

22강

예외처리

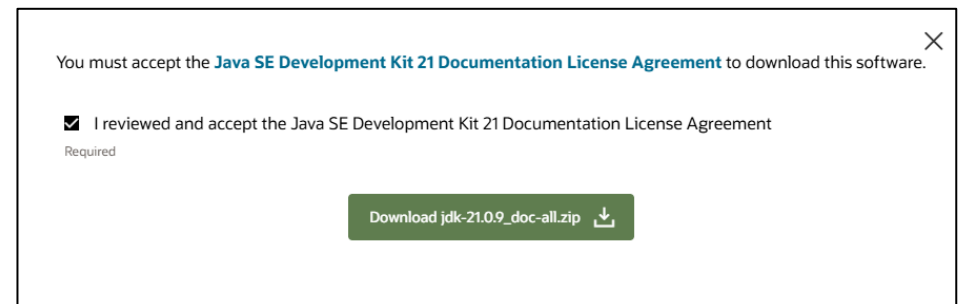
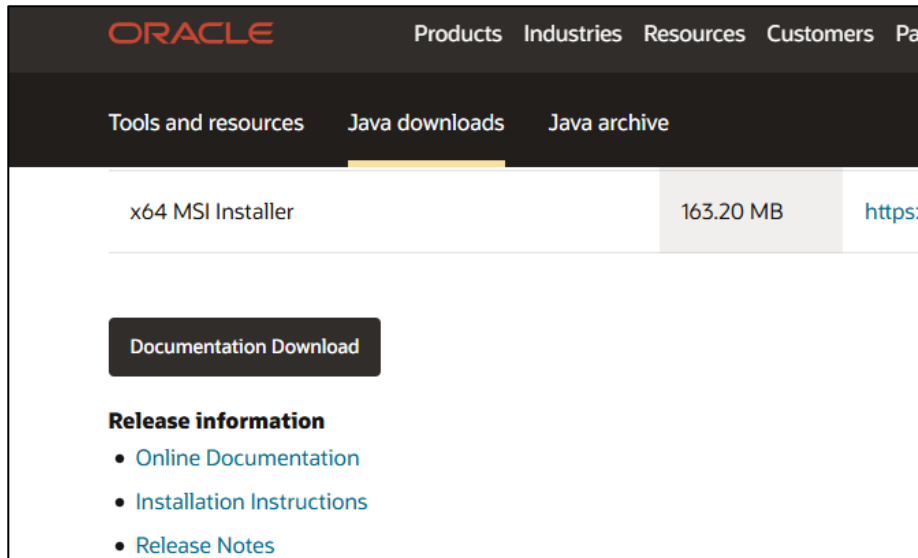




- 자바 API(Java API)는 자바를 사용하여 쉽게 구현할 수 있도록 한 **클래스 라이브러리의 집합**이다.
- 제공되는 유용한 클래스들에 대한 사용 방법과 정보를 문서화 해놓은 것이 자바 **API Document** 이다.
- 우리는 자바 프로그래밍 언어로 개발을 하기위해서는 자바 API 문서를 보고 그에 알맞게 사용할 수 있어야한다.

자바 API(**Application Programming Interface**) 문서 경로

1. <https://developer.oracle.com/kr/> 로 이동한다.
2. 상단 메뉴에서 '다운로드' 를 선택한다.
3. '모든 Java 다운로드'를 선택한다.
4. Java 에서 'Java(JDK) for Developers' 를 선택한다
5. 스크롤을 내려서 Java 21 을 선택한다.
6. 아래 화면의 하단에 있는 Documentation Download를 클릭한다.





Overview (Java SE 11 & JDK 11) x +

← → ↺ ① 파일 | D:/docs/api/index.html

Gmail YouTube 지도 Daum NAVER seezn(시존) - 즐겨... 승원프로그래밍강좌 자바예제 쿠팡! 위메프 하프클럽-패션전문... ZUM-실시간 뉴스...

OVERVIEW MODULE PACKAGE CLASS USE TREE DEPRECATED INDEX HELP

ALL CLASSES

Java® Platform, Standard Edition & Java Development Kit Version 11 API Specification

This document is divided into two sections:

- Java SE**
The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with `java`.
- JDK**
The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules whose names start with `jdk`.

All Modules Java SE JDK Other Modules

Module	Description
<code>java.base</code>	Defines the foundational APIs of the Java SE Platform.
<code>java.compiler</code>	Defines the Language Model, Annotation Processing, and Java Compiler APIs.
<code>java.datatransfer</code>	Defines the API for transferring data between and within applications.
<code>java.desktop</code>	Defines the AWT and Swing user interface toolkits, plus APIs for accessibility, audio, imaging, printing, and JavaBeans.
<code>java.instrument</code>	Defines services that allow agents to instrument programs running on the JVM.
<code>java.logging</code>	Defines the Java Logging API.
<code>java.management</code>	Defines the Java Management Extensions (JMX) API.
<code>java.management.rmi</code>	Defines the RMI connector for the Java Management Extensions (JMX) Remote API.
<code>java.naming</code>	Defines the Java Naming and Directory Interface (JNDI) API.
<code>java.net.http</code>	Defines the HTTP Client and WebSocket APIs.
<code>java.prefs</code>	Defines the Preferences API.
<code>java.rmi</code>	Defines the Remote Method Invocation (RMI) API.
<code>java.scripting</code>	Defines the Scripting API.
<code>java.se</code>	Defines the API of the Java SE Platform.
<code>java.security.jgss</code>	Defines the Java binding of the IETF Generic Security Services API (GSS-API).
<code>java.security.sasl</code>	Defines Java support for the IETF Simple Authentication and Security Layer (SASL).



➤ 자바 플랫폼의 3대 구성 요소

1. 자바 개발 키트(Java Development Kit, JDK)
2. 자바 가상 머신(Java Virtual Machine, JVM)
3. 자바 런타임 환경(Java Runtime Environment, JRE)은 자바 애플리케이션을 개발하고 실행하기 위한 자바 플랫폼의 3대 구성 요소

➤ JRE (Java Runtime Environment)

런타임 환경은 다른 소프트웨어를 실행하기 위해 고안되는 일종의 소프트웨어다. 자바용 런타임 환경인 **JRE**에는 자바 클래스 라이브러리(Java class libraries)와 자바 클래스 로더(Java class loader), 자바 가상 머신(Java Virtual Machine)이 포함된다

- 클래스 로더는 올바르게 클래스를 로드해 코어 자바 클래스 라이브러리에 연결하는 역할을 한다.
- **JVM**은 자바 애플리케이션이 디바이스 또는 클라우드 환경에서 실행되는 데 필요한 리소스를 확보하도록 보장하는 역할을 한다.
- **JRE**는 주로 다른 구성 요소의 컨테이너이며 각 구성 요소의 활동을 조율하는 역할을 한다.



➤ 오류란 무엇인가 ?

- 컴파일 오류(compile error) : 프로그램 코드 작성 중 발생하는 문법적 오류
- 실행 오류(runtime error) : 실행 중인 프로그램이 의도 하지 않은 동작을 하거나(bug) 프로그램이 중지되는 오류
- 실행 오류 시 비정상 종료는 서비스 운영에 치명적
- 오류가 발생할 수 있는 경우에 로그(log)를 남겨 추후 이를 분석하여 원인을 찾아야 함
- 자바는 예외 처리를 통하여 프로그램의 비정상 종료를 막고 log를 남길 수 있음



- 프로그램에 문제가 있는 것을 말하며, 예외로 인해 시스템 동작이 멈추는 것을 막는 것을 ‘예외처리’ 라고 한다.

소프트웨어적 오류

Exception

VS

Error

Error는 개발자가
대처할 수 있음.

Error는 개발자가
대처할 수 없음.

솔루션이 구동될 때 오류
메모리, JVM, 전원
물리적 오류

예외처리 최상위

Exception

Checked Exception

‘예외처리’를 반드시 해야하는 경우
(네트워크, 파일 시스템 등)

컴파일러가 예외를
처리하도록 JAVA가 강제로 지정 한다.

↳ 자동

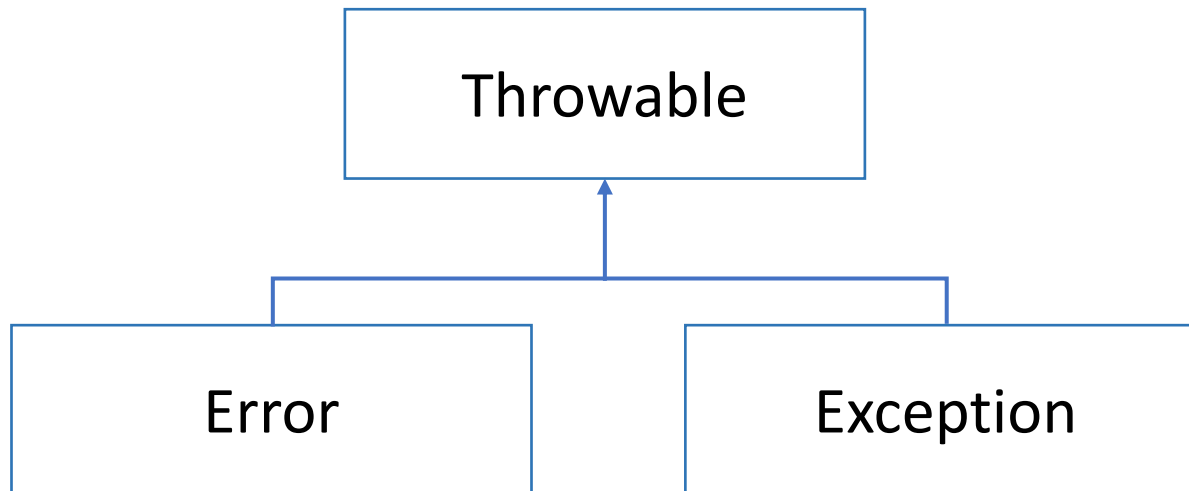
Unchecked Exception

‘예외처리’를 개발자의
판단에 맞기는 경우
(데이터 오류 등)

개발자가 알아서 예외처리를 지정 한다.



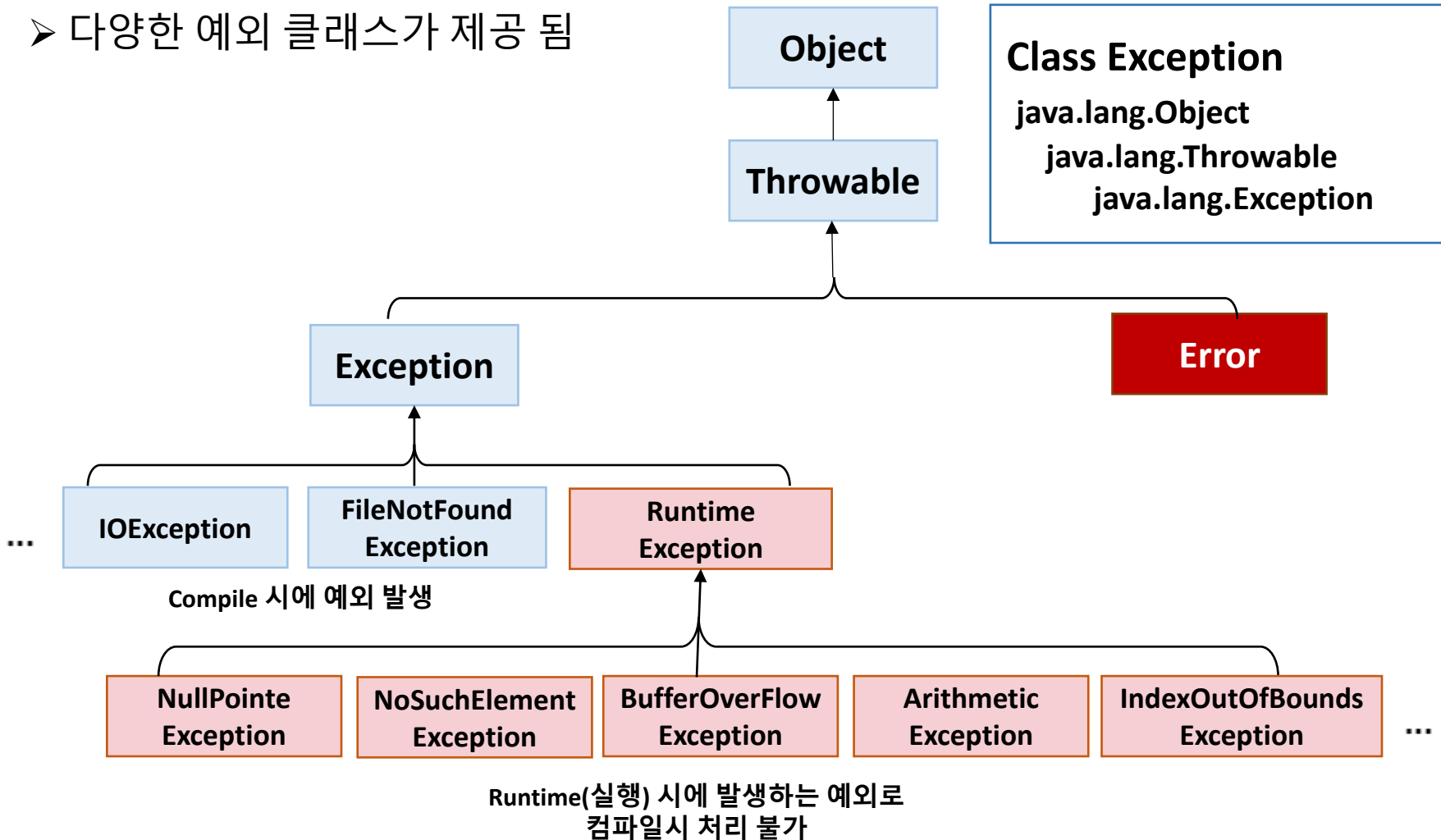
- 시스템 오류(error) : 가상 머신에서 발생, **프로그래머가 처리 할 수 없음**
동적 메모리 없는 경우, 스택 오버 플로우 등
- 예외(Exception) : **프로그램에서 제어 할 수 있는 오류**
읽어 들이려는 파일이 존재하지 않는 경우, 네트워크 연결이 끊어진 경우





➤ 모든 예외 클래스의 최상위 클래스는 Exception

➤ 다양한 예외 클래스가 제공 됨





예외 구문

이유

ArithmeticException

정수를 0으로 나눌 경우 발생

ArrayIndexOutOfBoundsException

배열의 범위를 벗어난 index를 접근할 시 발생

ClassCastException

변환할 수 없는 타입으로 객체를 반환 시 발생

NullPointerException

존재하지 않는 레퍼런스를 참조 할 때 발생

IllegalArgumentException

잘못된 인자를 전달 할 때 발생

IOException

입출력 동작 실패 또는 인터럽트 시 발생

OutOfMemoryException

메모리가 부족한 경우 발생

NumberFormatException

문자열이 나타내는 숫자와 일치하지 않는 타입의 숫자로 변환 시 발생



- 개발자가 예외처리하기 가장 쉽고, 많이 사용하는 방식이다.
- 프로그램 수행 도중 예외가 발생하면 프로그램이 오류에 의해 중지되거나 **catch 구문이 실행**된다

```
try {  
    예외가 발생할 수 있는 코드  
} catch (Exception e) {  
    예외가 발생했을 때 처리할 코드  
}
```

```
Ecexption BEFORE  
java.lang.ArithmeticException: / by zero  
Exception: / by zero    at lec26Pjt001.MainClass001.main(MainClass001.java:16)  
  
Ecexption AFTER
```

```
int i = 10;  
int j = 0;  
int r = 0;
```

```
System.out.println("Ecexption BEFORE");
```

```
try {  
    r = i / j;  
} catch (Exception e) {  
    e.printStackTrace();  
    String msg = e.getMessage();  
    System.out.println("Exception: " + msg);  
}
```

Try 안에서는 실행되지 않는다.

콘솔 창에 에러 출력하는 메서드

```
System.out.println("Ecexption AFTER");
```



```
public static void main(String[] args) {

    int[] arr = {1,2,3,4,5};

    for(int i=0; i<=5; i++) {
        System.out.println(arr[i]);
    }

}
```

RuntimeException 오류가
발생하여 시스템이 멈춘다.

Console x

<terminated> ArrayExceptionTest [Java Application] C:\Users\Admin\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64

```
1
2
3
4
5
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds
for length 5
    at exception.ArrayExceptionTest.main
    (ArrayExceptionTest.java:9)
```



```
public static void main(String[] args) {  
  
    int[] arr = {1,2,3,4,5};  
  
    try {  
        // try안의 코드가 수행되는 동안 오류가 발생한다면  
        for(int i=0; i<=5; i++) {  
            System.out.println(arr[i]);  
        }  
    } catch (ArrayIndexOutOfBoundsException e) {  
        // catch 구문안으로 들어와라  
        System.out.println(e.toString());  
    }  
    System.out.println("end");  
}
```

RuntimeException 오류가
발생하여도 시스템이 멈추지 않고
catch 구문과 End 까지 출력이 된다.

```
Console x  
<terminated> ArrayExceptionTest [Java Application] C:\Users\Admin\p2\pool\plugins\work  
1  
2  
3  
4  
5  
java.lang.ArrayIndexOutOfBoundsException:  
Index 5 out of bounds for length 5  
end
```



```
public static void main(String[] args) {  
  
    int[] arr = {1,2,3,4,5};  
  
    try {  
        // try안의 코드가 수행되는 동안 오류가 발생한다면  
        for(int i=0; i<=5; i++) {  
            System.out.println(arr[i]);  
        }  
    } catch (ArrayIndexOutOfBoundsException e) {  
        // catch 구문안으로 들어와라  
        System.out.println(e);  
        return;  
    } finally {  
        // try 구문이 수행이 되면 어떤 경우라도 무조건 수행되는 구문이다.  
        System.out.println("finally");  
    }  
  
    System.out.println("end");  
}
```

RuntimeException 오류가
발생하여도 멈추지 않고 finally는
무조건 출력된다.

```
Console x  
<terminated> ArrayExceptionTest [Java Application] C:\Users\Admin\p2\pool\plugins\org.e  
1  
2  
3  
4  
5  
java.lang.ArrayIndexOutOfBoundsException:  
Index 5 out of bounds for length 5  
finally
```



➤ Exception 및 하위 클래스를 이용해서 예외 처리를 다양하게 할 수 있다.

```
public class ExceptionTest {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int i=0;
        int j=0;
        int[] iArr = new int[5];
        List<Integer> list = new ArrayList<>();

        try {
            System.out.println("input i :");
            i = scan.nextInt();
            System.out.println("input j :");
            j = scan.nextInt();
            System.out.println(" i/j :"+(i/j));

            for(int k=0; k<6; k++) {
                System.out.println("iArr["+k+"]"+iArr[k]);
            }
            System.out.println("list size :"+list.size());

        } catch (InputMismatchException e) {
            e.printStackTrace();
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(e);
        } catch (java.lang.Exception e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("end");
        }
    }
}
```

// Error 스택 트레이스 출력

```
input i :
a
java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:939)
    at java.base/java.util.Scanner.next(Scanner.java:1594)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2258)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2212)
    at exception.ExceptionTest.main(ExceptionTest.java:21)
```

```
input i :
4
input j :
2
 i/j :2
iArr[0]0
iArr[1]0
iArr[2]0
iArr[3]0
iArr[4]0
java.lang.ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 5
```

```
Console x
<terminated> ExceptionTest (2) [Java Application]
input i :
4
input j :
0
/ by zero
end
```



- ▶ 예외 발생 시 예외 처리를 직접 하지 않고 호출한 곳으로 넘긴다.
- ▶ Throws를 사용하여 예외 처리 미루기
- ▶ 메서드 선언부에 throws 를 추가
- ▶ 예외가 발생한 메서드에서 예외 처리를 하지 않고 이 메서드를 호출한 곳에서 예외 처리를 한다는 의미
- ▶ Main() 에서 throws를 사용하면 가상머신에서 처리 됨

```
MainClass004 mainClass004 = new MainClass004();

try {
    mainClass004.firstMethod();
} catch (Exception e) {
    e.printStackTrace();
}
```



```
Console x
<terminated> Main (18) [Java Application] C:\Users\Admin\p2\pool\plugins\org.eclipse
java.lang.ArithmeticException: / by zero
```

```
public void firstMethod() throws Exception {
    secondMethod();
}

public void secondMethod() throws Exception {
    thirdMethod();
}

public void thirdMethod() throws Exception {
    System.out.println("10 / 0 = " + ( 10 / 0 ));
}
```